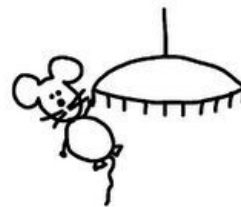


For personal use only,
please do not
distribute on
non-UBC domains.

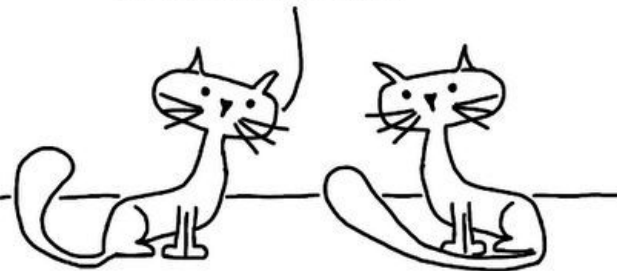
04a - Test Doubles

CPSC310

Nick Bradley & Kyle Chin



I've checked every square foot
in this house. I can confidently
say there are no mice here.



Absence of proof is not proof of absence.

- William Cowper

Learning Goals

By the end of the lecture you should be able to:

- Apply the *dependency inversion principle (DIP)* so tests can replace dependencies in the code under test.
- Design *stubs* and *spies* that stand in for real dependencies, to *control* what the code under test receives and *observe* what it sends.
- Apply the *Liskov substitution principle (LSP)* to decide whether a test double is a valid substitute.
- Decide when a test double is worth the cost, and when to use a real dependency instead.

Recall: Two Axes of Testability

Controllability

The degree to which a test can determine the inputs and state that the code under test depends on.

Observability

The degree to which a test can inspect the results and effects that the code under test produces.

What makes this code hard to test?

```
function printStatus(offeringId: string): void {  
    const offering = findOffering(offeringId);  
    if (offering === undefined) {  
        console.log(`no offering '${offeringId}'`);  
        return;  
    }  
    const offeringStart = Date.parse(SEASON[offering.season])  
    const daysLeft = offeringStart - Date.now()
```

What makes this code hard to test?

```
function printStatus(offeringId: string): void {  
  const offering = findOffering(offeringId);  
  if (offering === undefined) {  
    console.log(no offering '${offeringId}');  
    return;  
  }  
  const offeringStart = Date.parse(SEASON[offering.season])  
  const daysLeft = offeringStart - Date.now()
```

Observability: No way to
assert on log output

What makes this code hard to test?

```
function printStatus(offeringId: string): void {  
    const offering = findOffering(offeringId);  
    if (offering === undefined) {  
        console.log(no offering '${offeringId}');  
        return;  
    }
```

Observability: No way to
assert on log output

```
    const offeringStart = Date.parse(SEASON[offering.season])  
    const daysLeft = offeringStart - Date.now()
```

Controllability: No way to
set the date

Challenges...

Tests are particularly hard to write when the CUT is ...

- ***uncontrollable***: a bank, a clock
- ***unobservable***: you can't see what was sent
- ***unsafe or slow***: real money, the network

Improving testability

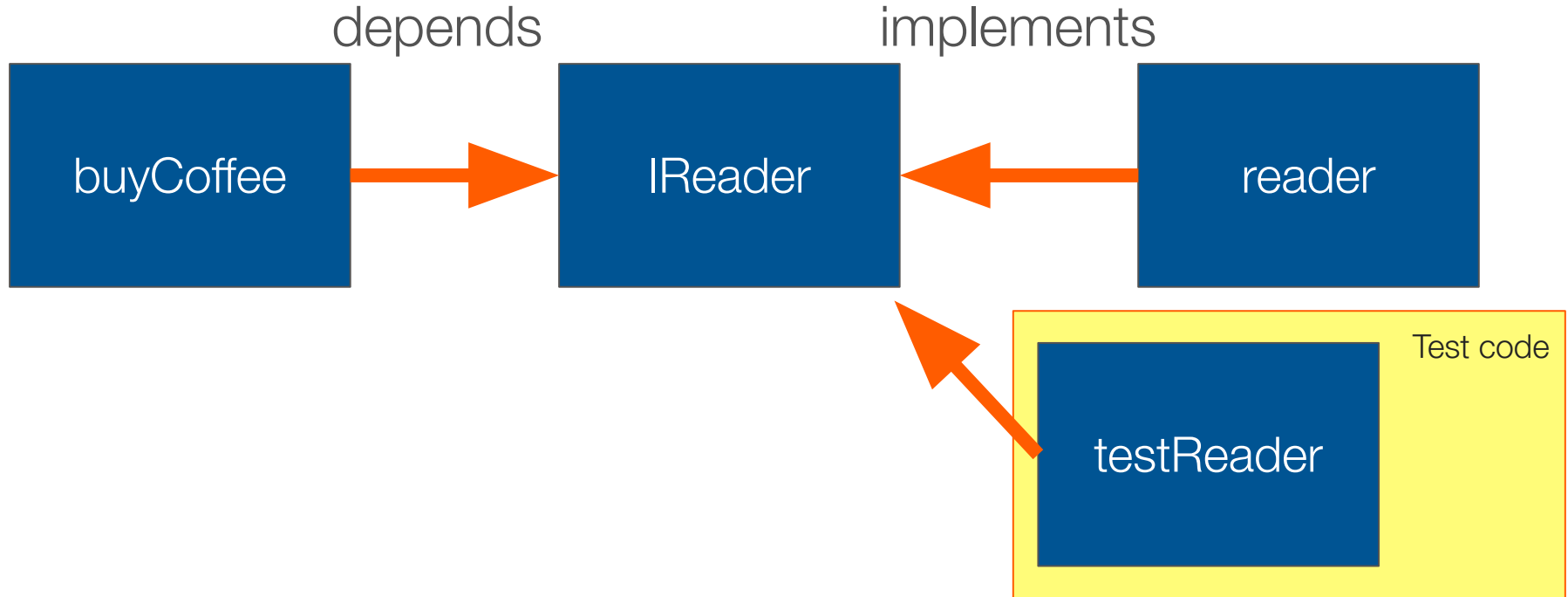
With the **Dependency Inversion Principle** (DIP)

“Depend upon *abstractions*,
do not depend upon
implementations.”



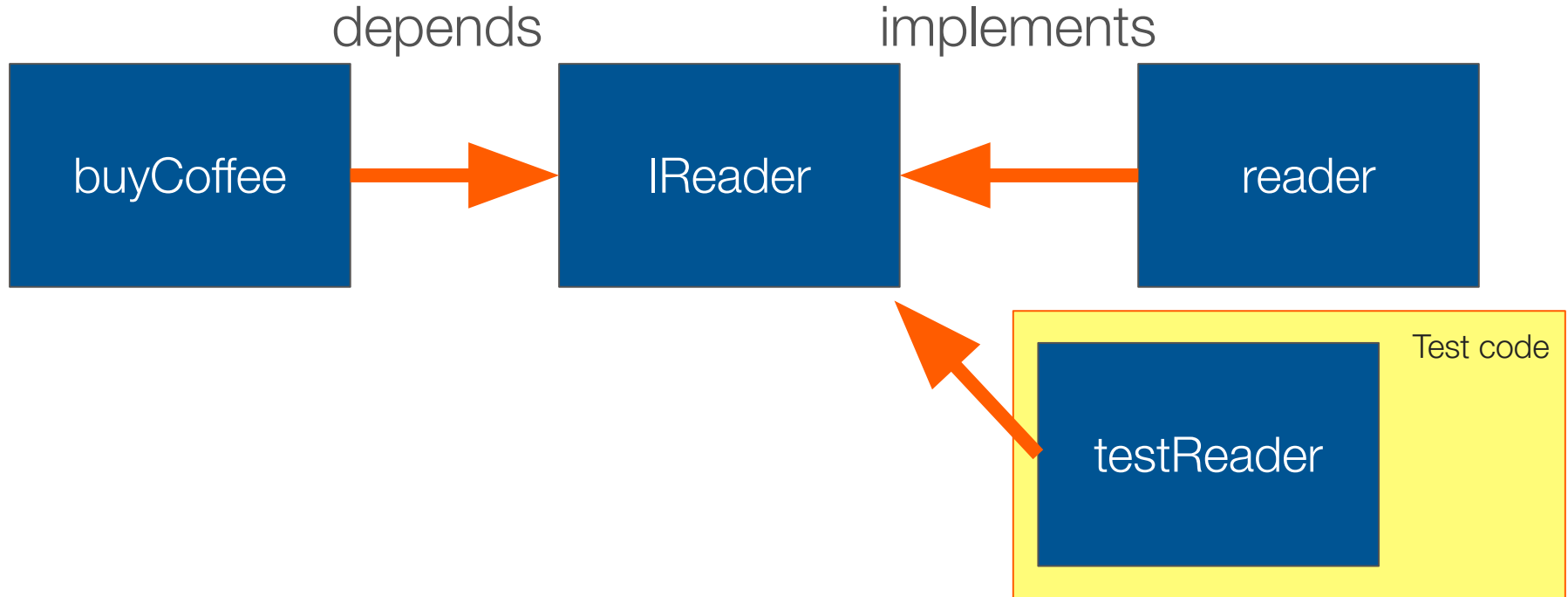
DIP helps with testability...

“Depend upon *abstractions*, do not depend upon *implementations*.”



...by enabling Test Doubles

testReader is called a *Test Double*



Test Doubles

- A test double stands in for a dependency that the code under test uses.
- A *stub* controls what the dependency gives back, like our approving and declining readers.
- A *spy* records what the code under test sent to it, like our recording reader.

When to use a test double

Use one when the real dependency is:

- uncontrollable: a bank, a clock
- unobservable: you can't see what was sent
- unsafe or slow: real money, the network

Otherwise, use the real thing. If it's fast, deterministic and has no side effects, don't double it.

Valid Test Doubles

Any object with a `charge` method that returns a boolean compiles as a `IReader`.

```
class FakeWorkingReader implements IReader {  
    charge = (n: number) => {  
        if (n < 0) throw new Error("Invalid charge!");  
        return true;  
    };  
}
```

Is compiling good enough?

- What does `IReader` guarantee `buyCoffee`? Where is that written down?
- Does this double keep that guarantee? The declined test fails: who's wrong, `buyCoffee` or the double?

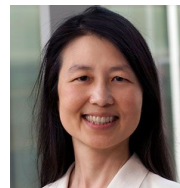
Liskov Substitution Principle

Subtype Requirement: Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should be true for objects y of type S where S is a subtype of T .

[Barbara Liskov and Jeanette Wing, A Behavioral Notion of Subtyping, ACM TOPLAS, 18:11-18:41, 1994.]



Barbara Liskov



Jeanette Wing

What is LSP?

Substitutability:

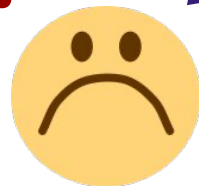
A subclass should not break the expectations set by its superclass!

```
class GoodPerson {  
    public freeBird(b: Bird) {  
        b.flyFromHeight(100);  
        // ... watch bird fly!  
    }  
}
```

```
class Bird {  
    // all heights work!  
    public flyFromHeight(height: number) {  
        // ... some implementation  
    }  
}
```



```
class Seagull extends Bird {  
    // all heights work!  
    public flyFromHeight(height: number) {  
        // ... some implementation  
    }  
}
```



```
class Penguin extends Bird {  
    // heights under 50cm only!  
    public flyFromHeight(height: number) {  
        // ... another implementation  
    }  
}
```



LSP Methods Rule

Substitutability:

A subclass
should not break
the expectations
set by its
superclass!

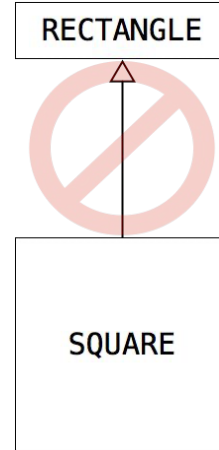
Precondition Rule

Preconditions should
not be strengthened
(for the same inputs)
OK to widen!

Postcondition Rule

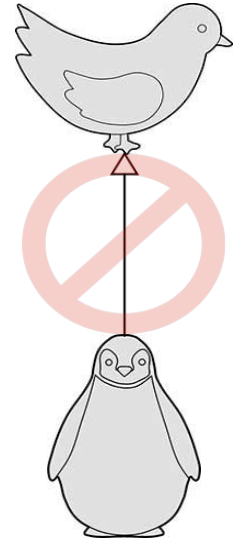
Postconditions should
not be weakened (for
the same inputs)
OK to narrow!

Resize only lets
you specify one
dimension.



Changing width will
also change height.
Bad: widened
postcondition!

Can fly.



Additional effect:
Penguin perishes
Bad: Widened
postcondition!

A young girl with red hair in pigtails, wearing a red dress, standing against a bright, hazy orange and yellow background.

Did you **create a test double?**

A close-up of Thanos, a purple-skinned character with a beard, looking slightly to the side with a serious expression.

Yes.

A young girl with red hair in pigtails, wearing a red dress, looking upwards with a slight smile against a bright, hazy orange and yellow background.

What did it cost?

A close-up of Thanos, a purple-skinned character with a beard, looking slightly to the side with a serious expression.

Everything

Recap: Testability with test doubles

- To test code that calls something, depend on an interface and pass the dependency in (DIP).
- **Test doubles** stand in for it. Stubs control what comes back, and spies record what goes out.
- A double has to keep the interface's contract, not just its shape (LSP).
- **The cost:** another layer; doubles can drift from the real thing & need to be updated when the interface changes. The real implementation still needs tests of its own.